Software Design and Architecture Assignment

Course: SWE 4601 - Software Design and Architecture

Student: Antara Arifa Mullick
Student ID: 210042132
Semester: 6th Semester
Date: September 26, 2025

Contents

1 Introduction **2**

2 SmartHire Project Enhancement **2**
2.1 Current System Overview 2
2.2 Clean Code Principles Implementation 2
2.2.1 Naming Conventions and Function Decomposition 2
2.2.2 Error Handling Enhancement 3
2.3 SOLID Principles Application 4
2.3.1 Single Responsibility Principle (SRP) 4
2.3.2 Open/Closed Principle (OCP) 5
2.4 Design Patterns Implementation 6
2.4.1 Factory Pattern for Resume Generation 6
2.4.2 Observer Pattern for Real-time Notifications 7
2.5 Architectural Patterns Implementation 8
2.5.1 MVVM Pattern for User Profile Management 8
2.6 Microservices Architecture with Message Passing 10
2.6.1 Service Decomposition 10
2.7 Database Design Enhancement 11
2.7.1 Event-Driven Database Operations 11
2.7.2 Database Replication Configuration 12

3 SWE 4601 Course Design Proposal **13**
3.1 Course Overview 13
3.2 Course Structure (14 Weeks) 13
3.3 Assessment Strategy 14
3.4 Active Learning Methods 14
3.5 Professional Development Integration 15
3.6 Expected Outcomes 15

1 Introduction

Software design and architecture form the foundation of scalable, maintainable, and robust software systems. The SmartHire project, developed during DP-I, serves as an excellent case study for applying advanced software engineering principles learned in SWE 4601. This report analyzes current limitations in the SmartHire codebase and proposes systematic improvements using clean code principles, SOLID design principles, architectural patterns, and modern software engineering practices.

Additionally, this report presents a comprehensive course design strategy for SWE 4601 that addresses common pedagogical challenges in software engineering education, emphasizing practical application over theoretical memorization.

2 SmartHire Project Enhancement

2.1 Current System Overview

SmartHire is a job searching and recruitment platform that connects job seekers with employers. The current implementation suffers from several architectural and code quality issues that limit its scalability and maintainability.

Current Architecture Problems:

- Monolithic frontend with tightly coupled React components
- Direct database access from controllers without proper abstraction
- Inconsistent naming conventions and poor function decomposition
- Lack of proper error handling and validation
- No clear separation of concerns between business logic and presentation

2.2 Clean Code Principles Implementation

2.2.1 Naming Conventions and Function Decomposition

Before (Problematic Code):

```
function handleStuff(data) {
    if(data.n && data.e && data.s) {
        // Complex nested logic spanning 50+ lines
        let result = doSomething(data);
        return generateStuff(result);
    }
    return null;
}
```

Listing 1: Problematic code example

After (Clean Code Applied):

```
class ResumeGenerator {
    constructor(userProfileService, pdfGeneratorService) {
        this.userProfileService = userProfileService;
        this.pdfGeneratorService = pdfGeneratorService;
        this.validator = new ProfileDataValidator();
    }
```

```
7
8       async generateProfessionalResume(userId) {
9           const userProfile = await this.getUserProfile(userId);
10          this.validateProfileCompleteness(userProfile);
11          const formattedData = this.formatProfileForResume(userProfile);
12          return await this.pdfGeneratorService.createDocument(
    formattedData);
13      }
14
15      validateProfileCompleteness(profile) {
16          if (!this.validator.hasRequiredFields(profile)) {
17              throw new IncompleteProfileError(
18                  "Profile must include name, email, and skills"
19              );
20          }
21      }
22
23      formatProfileForResume(profile) {
24          return {
25              personalInfo: this.extractPersonalInfo(profile),
26              professionalSummary: this.generateSummary(profile),
27              skillsSection: this.formatSkills(profile.skills),
28              experienceSection: this.formatExperience(profile.
    workHistory)
29          };
30      }
31  }
```

Listing 2: Clean code implementation

2.2.2 Error Handling Enhancement

Current Implementation:

```
1  // Poor error handling
2  async function searchJobs(criteria) {
3      const jobs = await database.find(criteria);
4      return jobs;
5  }
```

Listing 3: Poor error handling

Enhanced Implementation:

```
1  class JobSearchService {
2      constructor(jobRepository, logger) {
3          this.jobRepository = jobRepository;
4          this.logger = logger;
5      }
6
7      async searchJobs(searchCriteria) {
8          try {
9              this.validateSearchCriteria(searchCriteria);
10             const jobs = await this.jobRepository.findByCriteria(
    searchCriteria);
11             this.logger.info(`Job search completed: ${jobs.length}
    results found`);
12             return jobs;
13         } catch (ValidationError) {
```

```
            this.logger.warn('Invalid search criteria provided', { criteria: searchCriteria });
            throw new InvalidSearchCriteriaError('Please provide valid search parameters');
        } catch (DatabaseError) {
            this.logger.error('Database error during job search', { error });
            throw new JobSearchServiceError('Unable to complete search request');
        }
    }

    validateSearchCriteria(criteria) {
        const validator = new SearchCriteriaValidator();
        if (!validator.isValid(criteria)) {
            throw new ValidationError(validator.getErrors());
        }
    }
}
```

Listing 4: Enhanced error handling

2.3 SOLID Principles Application

2.3.1 Single Responsibility Principle (SRP)

Problem: Original JobComponent handled search, filtering, display, and user interactions.

Solution: Separate components with distinct responsibilities.

```
// Separate responsibility classes
class JobSearchEngine {
    constructor(jobRepository) {
        this.jobRepository = jobRepository;
    }

    async searchJobs(criteria) {
        return await this.jobRepository.findByCriteria(criteria);
    }
}

class JobFilterService {
    filterBySalaryRange(jobs, minSalary, maxSalary) {
        return jobs.filter(job =>
            job.salaryMin >= minSalary && job.salaryMax <= maxSalary
        );
    }

    filterByLocation(jobs, preferredLocations) {
        return jobs.filter(job =>
            preferredLocations.includes(job.location)
        );
    }

    filterBySkills(jobs, requiredSkills) {
        return jobs.filter(job =>
            requiredSkills.some(skill => job.requiredSkills.includes(skill))
```

```
        );
    }
}

class JobDisplayComponent extends React.Component {
    render() {
        const { jobs, onJobSelect } = this.props;
        return (
            <div className="job-listings">
                {jobs.map(job => (
                    <JobCard
                        key={job.id}
                        job={job}
                        onSelect={() => onJobSelect(job)}
                    />
                ))}
            </div>
        );
    }
}
```

Listing 5: SRP Implementation - Separate responsibility classes

2.3.2 Open/Closed Principle (OCP)

Implementation: Extensible skill assessment system.

```
abstract class SkillAssessmentStrategy {
    abstract calculateScore(responses: AssessmentResponse[]): number;
    abstract generateFeedback(score: number): string;
    abstract getPassingThreshold(): number;
}

class TechnicalSkillAssessment extends SkillAssessmentStrategy {
    calculateScore(responses: AssessmentResponse[]): number {
        let score = 0;
        responses.forEach(response => {
            score += this.evaluateTechnicalResponse(response);
        });
        return Math.min(score / responses.length * 100, 100);
    }

    generateFeedback(score: number): string {
        if (score >= 80) return "Excellent technical proficiency
   demonstrated";
        if (score >= 60) return "Good technical skills with room for
   improvement";
        return "Consider strengthening fundamental technical concepts";
    }

    getPassingThreshold(): number {
        return 70;
    }
}

class SoftSkillAssessment extends SkillAssessmentStrategy {
    calculateScore(responses: AssessmentResponse[]): number {
        return responses.reduce((total, response) => {
```

```
            return total + this.evaluateCommunicationClarity(response) +
                           this.evaluateLeadershipPotential(response);
        }, 0) / responses.length;
    }

    generateFeedback(score: number): string {
        if (score >= 85) return "Outstanding interpersonal and leadership qualities";
        if (score >= 70) return "Strong soft skills with collaborative potential";
        return "Focus on developing communication and teamwork abilities";
    }
}

// Assessment context that can work with any strategy
class AssessmentProcessor {
    constructor(strategy: SkillAssessmentStrategy) {
        this.strategy = strategy;
    }

    processAssessment(responses: AssessmentResponse[]): AssessmentResult {
        const score = this.strategy.calculateScore(responses);
        const feedback = this.strategy.generateFeedback(score);
        const passed = score >= this.strategy.getPassingThreshold();

        return new AssessmentResult(score, feedback, passed);
    }
}
```

Listing 6: OCP Implementation - Extensible skill assessment

2.4 Design Patterns Implementation

2.4.1 Factory Pattern for Resume Generation

```
class ResumeFactory {
    static createResumeBuilder(type: ResumeType, userProfile: UserProfile): ResumeBuilder {
        switch(type) {
            case ResumeType.TECHNICAL:
                return new TechnicalResumeBuilder(userProfile);
            case ResumeType.CREATIVE:
                return new CreativeResumeBuilder(userProfile);
            case ResumeType.EXECUTIVE:
                return new ExecutiveResumeBuilder(userProfile);
            case ResumeType.ACADEMIC:
                return new AcademicResumeBuilder(userProfile);
            default:
                return new StandardResumeBuilder(userProfile);
        }
    }
}

abstract class ResumeBuilder {
```

```
    protected userProfile: UserProfile;

    constructor(userProfile: UserProfile) {
        this.userProfile = userProfile;
    }

    abstract formatHeader(): ResumeSection;
    abstract formatSkillsSection(): ResumeSection;
    abstract formatExperience(): ResumeSection;
    abstract applyDesignTheme(): ResumeTheme;

    buildResume(): Resume {
        const sections = [
            this.formatHeader(),
            this.formatSkillsSection(),
            this.formatExperience()
        ];
        const theme = this.applyDesignTheme();
        return new Resume(sections, theme);
    }
}

class TechnicalResumeBuilder extends ResumeBuilder {
    formatSkillsSection(): ResumeSection {
        return new SkillsSection()
            .addProgrammingLanguages(this.userProfile.technicalSkills.languages)
            .addFrameworks(this.userProfile.technicalSkills.frameworks)
            .addTools(this.userProfile.technicalSkills.tools)
            .groupByCategory();
    }

    formatExperience(): ResumeSection {
        return new ExperienceSection()
            .emphasizeTechnicalAchievements()
            .highlightProjectContributions()
            .includeMetrics();
    }
}
```

Listing 7: Factory Pattern Implementation

2.4.2 Observer Pattern for Real-time Notifications

```
interface JobApplicationObserver {
    onApplicationStatusChanged(application: JobApplication): void;
    onInterviewScheduled(interview: Interview): void;
}

class JobApplicationSubject {
    private observers: JobApplicationObserver[] = [];

    addObserver(observer: JobApplicationObserver): void {
        this.observers.push(observer);
    }

    removeObserver(observer: JobApplicationObserver): void {
```

```
        this.observers = this.observers.filter(obs => obs !== observer);
    }

    notifyStatusChange(application: JobApplication): void {
        this.observers.forEach(observer => {
            observer.onApplicationStatusChanged(application);
        });
    }

    notifyInterviewScheduled(interview: Interview): void {
        this.observers.forEach(observer => {
            observer.onInterviewScheduled(interview);
        });
    }
}

class EmailNotificationService implements JobApplicationObserver {
    onApplicationStatusChanged(application: JobApplication): void {
        const emailTemplate = this.selectTemplate(application.status);
        this.sendEmail(application.applicantEmail, emailTemplate);
    }

    onInterviewScheduled(interview: Interview): void {
        const calendarInvite = this.createCalendarEvent(interview);
        this.sendCalendarInvite(interview.applicantEmail,
    calendarInvite);
    }
}

class InAppNotificationService implements JobApplicationObserver {
    onApplicationStatusChanged(application: JobApplication): void {
        const notification = new Notification(
            `Application status updated: ${application.status}`,
            application.applicantId
        );
        this.notificationQueue.add(notification);
    }
}
```

Listing 8: Observer Pattern Implementation

2.5 Architectural Patterns Implementation

2.5.1 MVVM Pattern for User Profile Management

```
class UserProfileViewModel {
    private userModel: UserModel;
    private validationService: ValidationService;

    // Observable properties
    public isLoading = observable(false);
    public validationErrors = observable([]);
    public profileData = observable({});

    constructor(userModel: UserModel, validationService:
    ValidationService) {
```

```
        this.userModel = userModel;
        this.validationService = validationService;
        this.loadUserProfile();
    }

    @action
    updateProfile = async (profileData: UserProfileData) => {
        this.isLoading.set(true);
        this.validationErrors.set([]);

        try {
            const validationResult = this.validationService.validate(
   profileData);
            if (!validationResult.isValid) {
                this.validationErrors.set(validationResult.errors);
                return;
            }

            await this.userModel.updateProfile(profileData);
            this.profileData.set(profileData);
            this.showSuccessMessage("Profile updated successfully");
        } catch (error) {
            this.handleUpdateError(error);
        } finally {
            this.isLoading.set(false);
        }
    }

    @computed
    get isProfileComplete(): boolean {
        const profile = this.profileData.get();
        return this.validationService.isCompleteProfile(profile);
    }
}

// React component using the ViewModel
const UserProfileView: React.FC = observer(() => {
    const viewModel = useUserProfileViewModel();

    return (
        <div className="profile-form">
            {viewModel.isLoading.get() && <LoadingSpinner />}

            <ProfileForm
                data={viewModel.profileData.get()}
                errors={viewModel.validationErrors.get()}
                onSubmit={viewModel.updateProfile}
                isComplete={viewModel.isProfileComplete}
            />
        </div>
    );
});
```

Listing 9: MVVM Pattern Implementation

2.6 Microservices Architecture with Message Passing

2.6.1 Service Decomposition

```
// User Management Service
class UserManagementService {
    constructor(userRepository: UserRepository, eventBus: EventBus) {
        this.userRepository = userRepository;
        this.eventBus = eventBus;
    }

    async createUser(userData: CreateUserRequest): Promise<User> {
        const user = await this.userRepository.create(userData);

        // Publish event for other services
        await this.eventBus.publish('user.created', {
            userId: user.id,
            email: user.email,
            skills: user.skills,
            preferences: user.jobPreferences,
            timestamp: new Date()
        });

        return user;
    }

    async updateUserSkills(userId: string, skills: Skill[]): Promise<
    void> {
        await this.userRepository.updateSkills(userId, skills);

        await this.eventBus.publish('user.skills.updated', {
            userId,
            skills,
            timestamp: new Date()
        });
    }
}

// Job Recommendation Service (Separate Microservice)
class JobRecommendationService {
    constructor(
        recommendationEngine: RecommendationEngine,
        eventBus: EventBus,
        cacheService: CacheService
    ) {
        this.recommendationEngine = recommendationEngine;
        this.eventBus = eventBus;
        this.cacheService = cacheService;
        this.setupEventHandlers();
    }

    private setupEventHandlers(): void {
        this.eventBus.subscribe('user.created', this.handleUserCreated.
    bind(this));
        this.eventBus.subscribe('user.skills.updated', this.
    handleSkillsUpdated.bind(this));
        this.eventBus.subscribe('job.posted', this.handleJobPosted.bind
    (this));
```

```
    }

    async handleUserCreated(event: UserCreatedEvent): Promise<void> {
        const initialRecommendations = await this.recommendationEngine
            .generateInitialRecommendations(event.userId, event.skills)
    ;

        await this.cacheService.store(
            `recommendations:${event.userId}`,
            initialRecommendations,
            { ttl: 3600 } // 1 hour cache
        );
    }
}
```

Listing 10: Microservices Implementation

2.7 Database Design Enhancement

2.7.1 Event-Driven Database Operations

```
class JobApplicationEventHandler {
    constructor(
        applicationRepository: ApplicationRepository,
        eventRepository: EventRepository,
        notificationService: NotificationService
    ) {
        this.applicationRepository = applicationRepository;
        this.eventRepository = eventRepository;
        this.notificationService = notificationService;
    }

    async handleJobApplication(applicationData: JobApplicationData):
    Promise<void> {
        const session = await this.database.startSession();

        try {
            await session.withTransaction(async () => {
                // Create application record
                const application = await this.applicationRepository.
    create(
                    applicationData,
                    { session }
                );

                // Create audit event
                await this.eventRepository.create({
                    type: 'APPLICATION_SUBMITTED',
                    entityId: application.id,
                    userId: applicationData.applicantId,
                    metadata: {
                        jobId: applicationData.jobId,
                        recruiterId: applicationData.recruiterId,
                        source: applicationData.source
                    },
                    timestamp: new Date()
                }, { session });
```

```
35
36                  // Update job statistics
37                  await this.updateJobStatistics(
38                      applicationData.jobId,
39                      { session }
40                  );
41
42                  // Schedule notifications (outside transaction)
43                  this.scheduleNotifications(application);
44              });
45          } finally {
46              await session.endSession();
47          }
48      }
49  }
```

Listing 11: Event-driven database operations

2.7.2 Database Replication Configuration

```
1  // MongoDB Replica Set Configuration
2  const replicationConfig = {
3      _id: 'smartHireReplica',
4      members: [
5          {
6              _id: 0,
7              host: 'primary.smarthire.com:27017',
8              priority: 2,
9              votes: 1
10         },
11         {
12             _id: 1,
13             host: 'secondary1.smarthire.com:27017',
14             priority: 1,
15             votes: 1
16         },
17         {
18             _id: 2,
19             host: 'secondary2.smarthire.com:27017',
20             priority: 0,
21             votes: 1,
22             hidden: true // For analytics and backups
23         }
24     ],
25     settings: {
26         chainingAllowed: true,
27         heartbeatIntervalMillis: 2000,
28         heartbeatTimeoutSecs: 10,
29         electionTimeoutMillis: 10000
30     }
31 };
32
33 // Database Service with Read Preference Configuration
34 class DatabaseService {
35     constructor() {
36         this.mongoClient = new MongoClient(connectionString, {
37             replicaSet: 'smartHireReplica',
```

```
            readPreference: 'secondaryPreferred',
            readConcern: { level: 'majority' },
            writeConcern: { w: 'majority', wtimeout: 5000 }
        });
    }

    // Read operations preferably from secondaries
    async findJobs(criteria: JobSearchCriteria): Promise<Job[]> {
        return await this.collection('jobs')
            .find(criteria)
            .readPref('secondary')
            .toArray();
    }

    // Write operations to primary with write concern
    async createJobApplication(applicationData: JobApplicationData):
   Promise<JobApplication> {
        return await this.collection('applications')
            .insertOne(applicationData, {
                writeConcern: { w: 'majority', wtimeout: 5000 }
            });
    }
}
```

Listing 12: MongoDB Replica Set Configuration

3 SWE 4601 Course Design Proposal

3.1 Course Overview

The SWE 4601 course is designed to close the gap between theory and practice in software engineering. It emphasizes hands-on coding, scenario-based learning, and continuous feedback to prepare students for real-world challenges.

Core Learning Objectives:

1. Apply clean code and refactoring techniques in real codebases
2. Implement SOLID principles and design patterns effectively
3. Design scalable software architectures for different system sizes
4. Understand and apply database replication strategies
5. Communicate and defend technical decisions in teams

3.2 Course Structure (14 Weeks)

The course runs for 14 weeks, divided into focused modules that build on each other.

- **Weeks 1-2: Clean Code Foundations** Students practice identifying bad coding practices, refactoring, and code reviews. Activities: messy code challenges, naming conventions, before/after comparisons.
- **Weeks 3-4: SOLID Principles** Introduction through "violation-first" learning. Students spot principle violations and redesign simple systems. Activities: SOLID detective game, redesign challenges, team presentations.

- **Weeks 5-7: Design Patterns Laboratory** Explore key design patterns through discovery, coding labs, and small framework projects. Activities: Factory, Builder, Observer, Strategy patterns; anti-pattern analysis.
- **Weeks 8-9: Software Architecture Patterns** Compare architectural choices (monolith, modular monolith, microservices). Activities: scalability workshops, ADR documentation, system evolution exercises.
- **Weeks 10-12: Microservices & Distributed Systems** Learn decomposition, communication, and fault tolerance. Activities: microservice labs with Docker, message queue experiments, failure simulations.
- **Weeks 13-14: Database Design & Replication** Focus on performance-driven learning with replication and CAP theorem trade-offs. Activities: replication setup labs, performance benchmarking, consistency vs availability discussions.

3.3 Assessment Strategy

The course follows the official assessment structure while adding supportive elements to strengthen learning.

- **Official Assessment Scheme:**
  - 3 Quizzes: 15 marks each
  - Mid-term Examination: 75 marks
  - Final Examination: 150 marks
  - Attendance: 30 marks
- **Supportive Assessment Activities (for practice and bonus):**
  - **Pop Quizzes:** Short, unannounced quizzes to reinforce concepts. Correct answers can earn up to 5 bonus marks.
  - **Assignments:** Practical coding or design tasks given periodically. These help students prepare for quizzes and exams by applying concepts in practice.

3.4 Active Learning Methods

- **Live Coding Sessions:** Instructors code with students, showing problem-solving in real time
- **Code Reviews:** Teams critique code samples, practicing professional review skills
- **Scenario-Based Practice:** Real-world inspired coding challenges each week
- **Group Work:** Team discussions and collaborative projects to simulate workplace dynamics

3.5 Professional Development Integration

- Guest talks from software architects and industry experts
- Open-source contribution requirements to build real experience
- Mock technical interviews and design Q&A sessions
- Emphasis on technical documentation and communication

3.6 Expected Outcomes

By the end of the course, students will:

- Write clean, maintainable code following best practices
- Apply design principles and patterns appropriately
- Evaluate trade-offs and select suitable architectural patterns
- Understand system scalability and database replication issues
- Communicate effectively in technical discussions and team projects